

California State University Long Beach
Computer Engineering and Computer Science Department
CECS 440 – Computer Architecture
Lab 8 – Memory Access (MEM) stage

Objectives:

- **Implement the Memory Access (MEM) stage** of a single-cycle MIPS processor in Verilog.
- **Use control signals** (*MemWriteM*, *Branch*) to manage memory operations and branch logic.
- **Use ZeroM** and control signal **BranchM** to generate control signal **PCSrcM**.
- **Connect and verify data paths** for *ALUOutM*, *WriteDataM*, and *ReadDataM* between the Execute (EX) and Write Back (WB) stages.
- **Simulate and confirm correct memory read/write behavior** and branch operation using **Xilinx Vivado** behavioral simulation with a Verilog testbench.

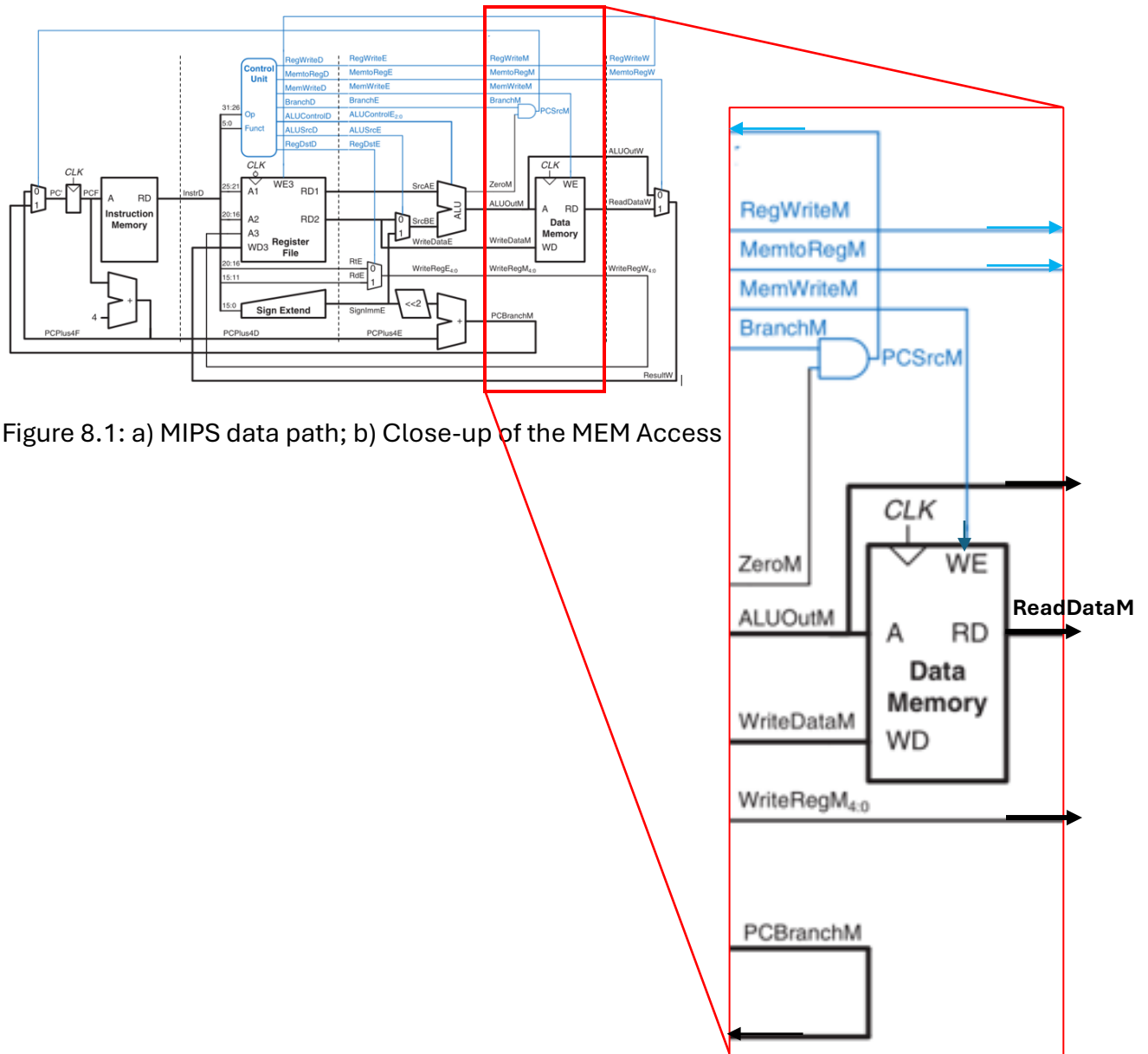
Background:

In the single-cycle MIPS processor, instruction execution occurs in five stages. The **Memory Access (MEM) stage** is responsible for interacting with **data memory (DMEM)**. During this stage, memory can be **written to** for a *store word (sw)* instruction or **read from** for a *load word (lw)* instruction. This stage also includes logic to determine whether a **branch** should be taken.

The key signals used in the MEM stage are described below:

- **ALUOutM:** The address used to either write data to memory (for *sw*) or read data from memory (for *lw*).
- **WriteDataM:** The data to be written to DMEM on the positive clock edge when **MemWriteM** (connected to Write Enable WE) is set to 1.
- **ReadDataM:** The data read from DMEM
- **PCSrcM:** PCSrcM is asserted (equal to 1) when BranchM and ZeroM are equal to 1.
- **RegWriteM** , **MemtoRegM**, **WriteReg**, and **PCBranchM** are the same as in the previous (Execute) stage.

In summary, the MEM stage handles memory read/write operations and participates in branch decision logic, preparing data and control signals for the subsequent **Write Back (WB)** stage.



Steps:

1. Create Data Memory in a file named **dmem.v** and name the module **dmem**, as per Figure 8.2.

```
1 module dmem(clk, we, a, wd, rd);
2     inout clk, we;
3     input [31:0] a, wd;
4     output [31:0] rd;
5
6     reg [31:0] RAM[63:0];
7
8     assign rd = ;
9
10    always @(posedge clk)
11        if (we) RAM[a[31:2]] <= wd;
12 endmodule
```

Figure 8.2: Code for the dmem module

2. Use the block diagram on page 1 and complete the module skeleton given in Figure 8.3 to create the mem stage:

```
1 module MEM_Stage (
2     input wire clk,
3     input wire regwritee,
4     input wire memtorege,
5     input wire memwritem,
6     input wire branchm,
7     input wire zerom,
8     input wire [31:0] aluoute,
9     input wire [31:0] writedatam,
10    input wire [4:0] writerege,
11    output wire pcsrcm,
12    input wire [31:0] pcbranche,
13    output wire regwritem,
14    output wire memtoregm,
15    output wire [31:0] aluoutm,
16    output wire [31:0] readdatam,
17    output wire [4:0] writeregm,
18    output wire [31:0] pcbranchm
19 );
20
21 assign regwritem = ;
22 assign memtoregm = ;
23 assign pcsrcm = ;
24 assign aluoutm = ;
25 assign writeregm = ;
26 assign pcbranchm = ;
27
28 // Data memory (assumes rd is an output of dmem)
29 dmem dmem (
30
31 );
32
33 endmodule
```

Figure 8.3: Skeleton of the code for the MEM stage module

3. Test your design to ensure it works correctly using the Verilog test fixture code in Figure 8.4. Correct test results are shown on Figure 8.5.

```
3 module t_MEM_Stage();
4     reg clk, regwritee, memtorege, memwritem, branchm, zerom;
5     reg [31:0] aluoute, pcbranche, writedatam;
6     reg [4:0] writerege;
7     wire pcsrcm, regwritem, memtoregm;
8     wire [31:0] aluoutm, readdatam, pcbranchm;
9     wire [4:0] writeregm; integer i;
10
11     MEM_Stage dut(    );
12
13     always #5 clk = ~clk; // clock pulse generation
14     initial begin clk = 1; // $dumpfile("dump.vcd"); $dumpvars(0,dut);
15         @(negedge clk) rndctrlsig(); showmem(); checksignals();
16         memwritem = 1; aluoute = 32'h0; writedatam = 32'h01010101;
17         @(negedge clk) rndctrlsig(); showmem(); checksignals();
18         memwritem = 1; aluoute = 32'h4; writedatam = 32'h12121212;
19         @(negedge clk) rndctrlsig(); showmem(); checksignals();
20         memwritem = 1; aluoute = 32'h8; writedatam = 32'h23232323;
21         #10 showmem(); $finish;
22     end
23     task checksignals; begin @(posedge clk) #1
24         if(regwritem != regwritee || memtoregm != memtorege ||
25             pcsrcm != (branchm & zerom) || aluoutm != aluoute ||
26             writeregm != writerege || pcbranchm != pcbranche)
27             begin $display("Test Failed: Control Signals faulty"); $finish; end
28         else begin $display("Control signals OK"); end
29     end endtask
30     task rndctrlsig;
31         {regwritee, memtorege, branchm, zerom, writerege, pcbranche} = $random;
32     endtask
33     task showmem; begin @(posedge clk) #1
34         for(i = 0; i < 4; i = i + 1)
35             $display("dmem[%h] = %h", i, dut.dmem.RAM[i]);
36         end endtask
37 endmodule
```

Figure 8.4: Testbench for the MEM_Stage module

```
dmem[00000000] = xxxxxxxx
dmem[00000001] = xxxxxxxx
dmem[00000002] = xxxxxxxx
dmem[00000003] = xxxxxxxx
Control signals OK
dmem[00000000] = 01010101
dmem[00000001] = xxxxxxxx
dmem[00000002] = xxxxxxxx
dmem[00000003] = xxxxxxxx
Control signals OK
dmem[00000000] = 01010101
dmem[00000001] = 12121212
dmem[00000002] = xxxxxxxx
dmem[00000003] = xxxxxxxx
Control signals OK
dmem[00000000] = 01010101
dmem[00000001] = 12121212
dmem[00000002] = 23232323
dmem[00000003] = xxxxxxxx
Done
```

Figure 8.5: Correct test results

Submission:

Once you have verified that your project is functioning correctly, copy the contents of **MEB_Stage module (design.v)**, **dmem.v** and **testbench_tb.v** into a single text file named **lab8-MEM.txt**. Submit **this file** along with your **lab report** on Canvas.

Write your report using the **Lab Report Template** available on Canvas. Be sure to include:

- Screenshots of your code
- Console output
- One waveform screenshot showing a few selected test cases

Note: Keep all lab files, as they will be needed for future labs.

Created by Josh Hayter, Jelena Trajkovic and Bronson Garel (Copyright © 2025)